

Ejercicio 3.5

Introducción al Deep Learning mediante Google Quick, Draw! y TensorFlow Playground

ALUMNO

Miguel Jericó

FORMACIÓN

INAEM

Análisis de Datos e IA

ENTORNO

Web

Quick, Draw! ·
TensorFlow Playground

BLOQUE

Deep Learning

Redes neuronales y
reconocimiento de
patrones

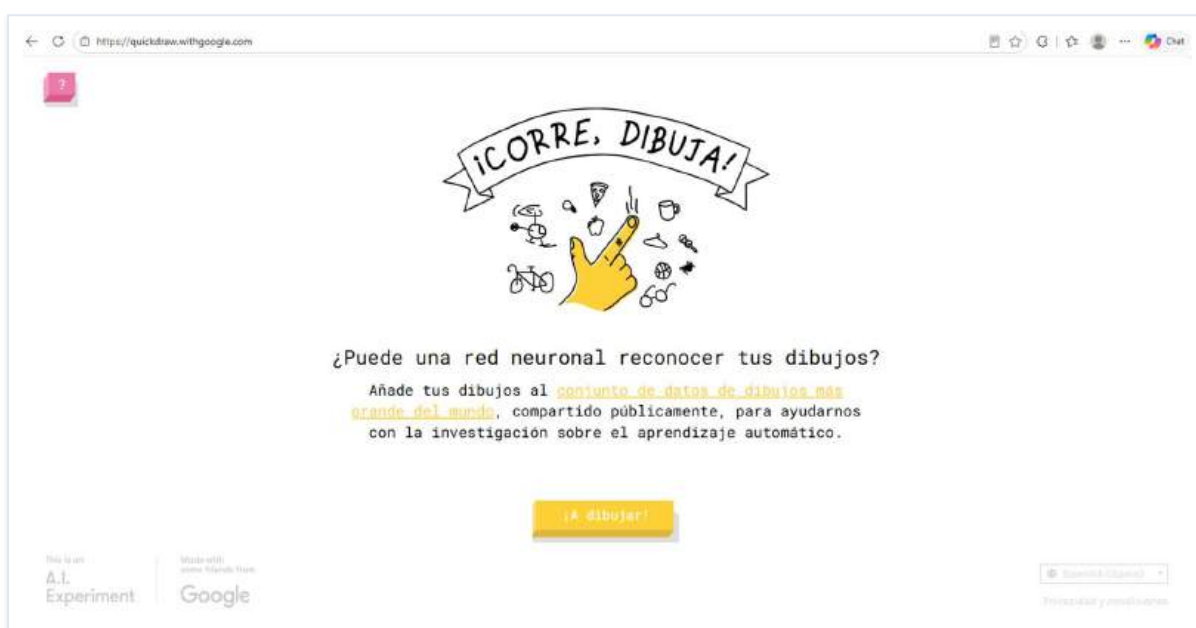
RESUMEN DEL EJERCICIO

Práctica de introducción al Deep Learning dividida en dos partes. La primera explora **Google Quick, Draw!**, un modelo de inferencia ya entrenado con más de 50 millones de dibujos, para comprender de forma intuitiva cómo una red neuronal reconoce patrones trazados a mano. La segunda utiliza **TensorFlow Playground**, un entorno visual de entrenamiento, para construir redes neuronales desde cero, modificar su arquitectura (capas ocultas y neuronas), probar distintas funciones de activación (ReLU, Tanh, Sigmoid) y observar en directo cómo desciende el error y se adapta la frontera de decisión a medida que avanzan las épocas.

01

Acceder a la página web

Se accede a la web oficial de Google Quick, Draw! (quickdraw.withgoogle.com), un experimento de Inteligencia Artificial que reconoce en tiempo real los dibujos trazados a mano por el usuario. En la pantalla inicial aparece el lema "¡Corre, dibuja!" y un botón amarillo que invita a comenzar la partida. En este punto la red neuronal del juego ya está entrenada y lista para inferir qué dibuja el usuario a partir de los trazos que introduzca.



Captura 1 • Pantalla de bienvenida de Google Quick, Draw!, con el lema "¡Corre, dibuja!" y el botón amarillo para iniciar la partida.

02

Completar una partida de 6 dibujos

Se pulsa el botón **▶ A dibujar** y comienza la partida. El sistema propone seis conceptos aleatorios (por ejemplo: arcoíris, corbata, arbusto, etc.) y dispone de 20 segundos para dibujar cada uno mientras la IA intenta adivinar en directo lo que se está trazando. No importa qué conceptos aparezcan, el objetivo es simplemente completar la secuencia y observar el comportamiento del reconocimiento.

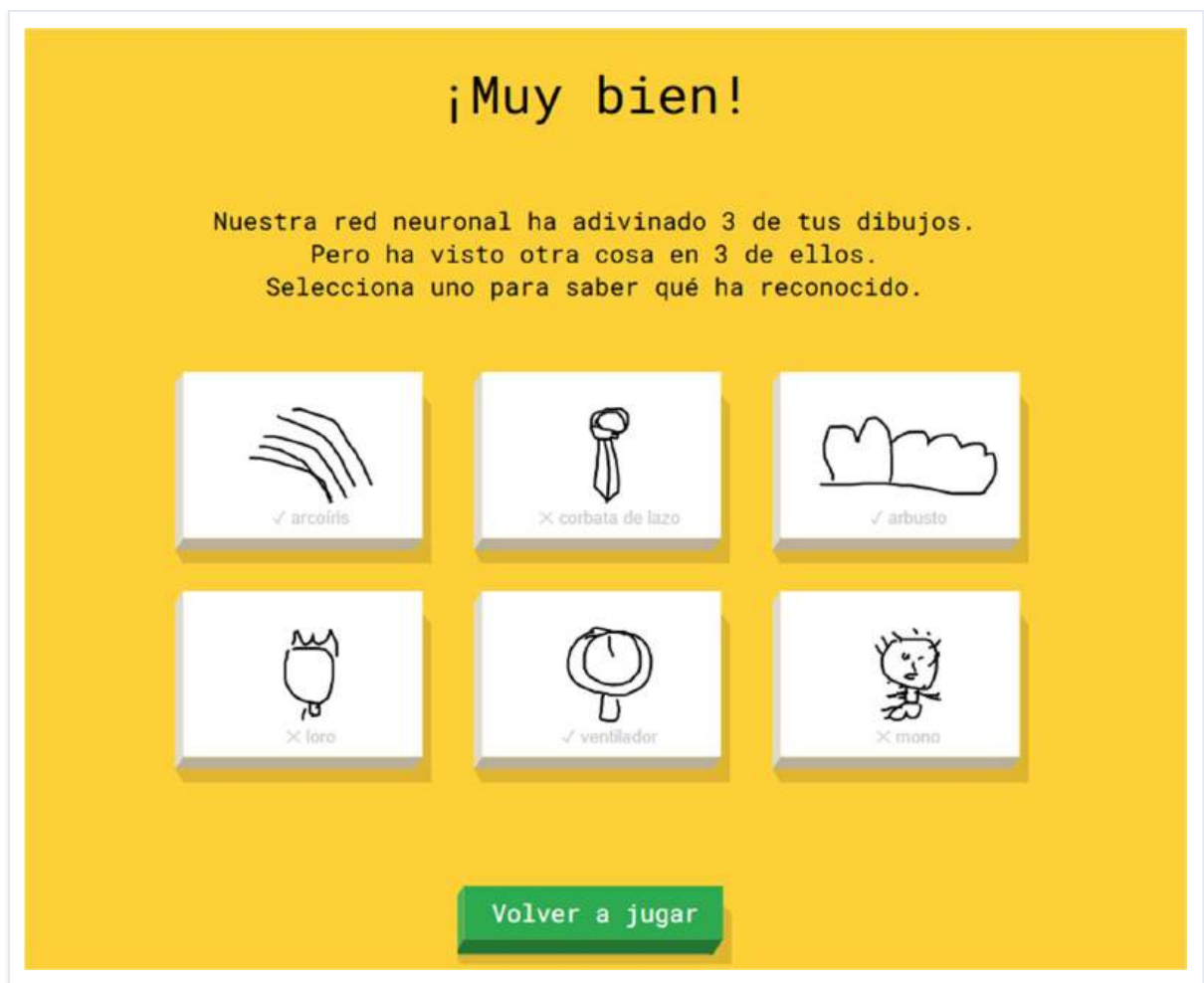


Captura 2 • Pantalla intermedia de la partida: se muestra el concepto a dibujar ("Dibujo 1/6") y el cronómetro de 20 segundos en la esquina superior derecha.

03

Observar el resultado final de la partida

Al terminar la partida aparece un resumen con los seis dibujos realizados. Para cada uno se indica si la IA lo ha reconocido correctamente, lo ha confundido con otro concepto o no ha logrado identificarlo. Es interesante comparar los dibujos acertados con los fallidos y analizar por qué el modelo ha podido equivocarse: trazos ambiguos, perspectivas inusuales, falta de contexto visual, etc.



Captura 3 • Resultado final de la partida: mensaje "¡Muy bien!" y los seis dibujos realizados, con la marca verde de acierto o la etiqueta del concepto que la IA creyó ver cuando falló.

04

Completar la tabla de reconocimiento

A partir de los resultados del paso anterior, se rellena una tabla registrando si la IA reconoció o no cada uno de los seis dibujos y el tiempo aproximado que tardó en hacerlo. Esta información permite cuantificar el comportamiento del modelo y compararlo con la percepción subjetiva del usuario.

Dibujo	¿Lo reconoció?	¿Cuánto tardó aproximadamente?
1	Sí	5 segundos
2	No	-
3	Sí	5 segundos
4	No	-
5	Sí	5 segundos
6	No	-

05

Reflexionar sobre el comportamiento de la IA

Tras jugar una partida, se plantean tres preguntas para reflexionar sobre qué está haciendo realmente la Inteligencia Artificial cuando "adivina" un dibujo. Las respuestas sirven de puente conceptual entre la experiencia de usuario y el funcionamiento interno de un modelo de Deep Learning.

1. ¿Crees que la IA entiende realmente el dibujo o reconoce patrones aprendidos?

La IA no entiende semánticamente qué es el objeto (carece de consciencia o concepto abstracto sobre lo que es un "gato" o una "silla"). Lo que hace es estrictamente reconocer patrones matemáticos aprendidos. Durante su entrenamiento, la red neuronal ha procesado millones de trazos y ha ajustado sus pesos para identificar características geométricas (líneas, curvas, ángulos, velocidad del trazo). Cuando dibujas, la IA convierte tus líneas en datos numéricos y calcula la probabilidad estadística de que esos vectores coincidan con una de las categorías que ya conoce.

2. ¿Por qué crees que unas veces acierta y otras no?

Acierta cuando tu forma de dibujar coincide con el "estándar" o promedio geométrico de los miles de dibujos con los que fue entrenada para esa clase. Falla por varias razones:

- **Ambigüedad del trazo:** si tu dibujo de un círculo con dos líneas puede ser matemáticamente tanto una "manzana" como un "reloj", la IA puede inclinarse por la opción incorrecta.
- **Perspectiva inusual:** si la IA fue entrenada mayoritariamente con dibujos de bicicletas vistas de lado, fallará si intentas dibujarla vista desde arriba o de frente.
- **Falta de contexto:** a diferencia de los humanos, la red neuronal solo ve píxeles y vectores, por lo que no puede usar el sentido común para deducir formas incompletas que se salen de su patrón estadístico.

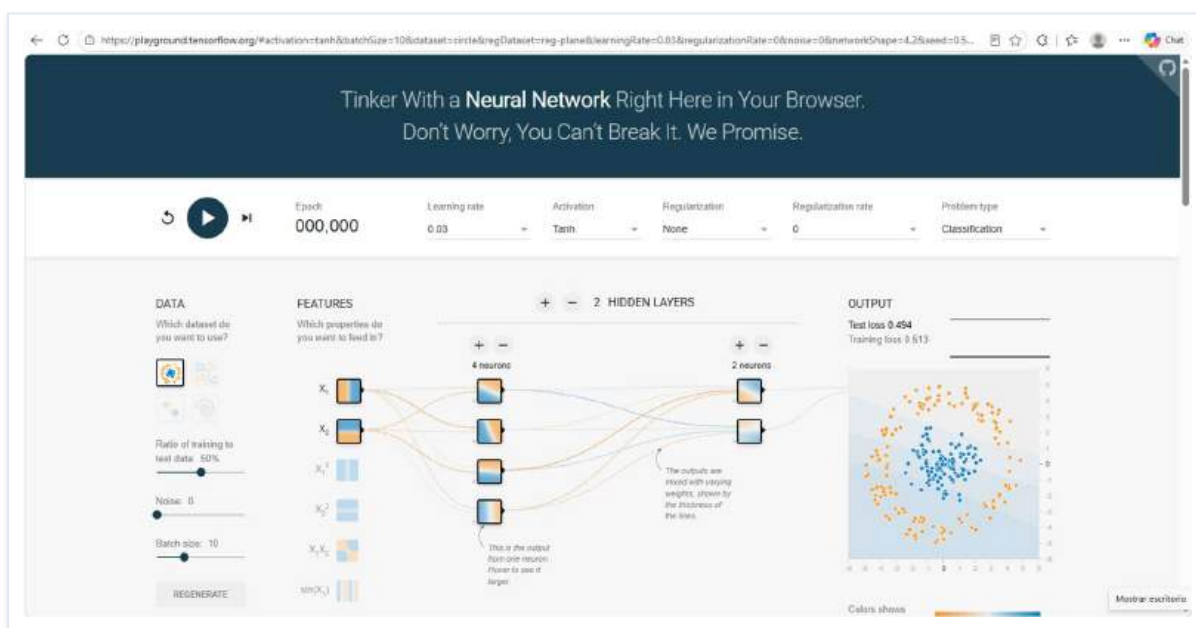
3. ¿Con cuántos dibujos crees que ha sido entrenada?

Los modelos de Deep Learning requieren un volumen de datos masivo para poder generalizar correctamente (Big Data). El modelo detrás de Quick, Draw! fue entrenado con un dataset gigantesco que contiene más de 50 millones de dibujos aportados por usuarios de todo el mundo, como indica la propia página. Esto es lo que le permite reconocer tantas variaciones de un mismo objeto (por ejemplo, personas de distintas culturas dibujando una "casa" de formas diferentes).

02 PARTE 2 TensorFlow Playground

06 Acceder a TensorFlow Playground

Se accede a **playground.tensorflow.org**, un entorno visual interactivo desarrollado por el equipo de TensorFlow que permite experimentar con redes neuronales poco profundas directamente en el navegador, sin escribir código. Al abrir la página aparece una red neuronal por defecto, ya configurada con un conjunto de datos de clasificación binaria y varias capas ocultas. La pantalla se divide en cuatro zonas: a la izquierda los **controles** (dataset, ratio de aprendizaje, función de activación, número de capas y neuronas); en el centro la **arquitectura de la red**; a la derecha el **resultado** (frontera de decisión, gráfica de error y distribución de pesos).

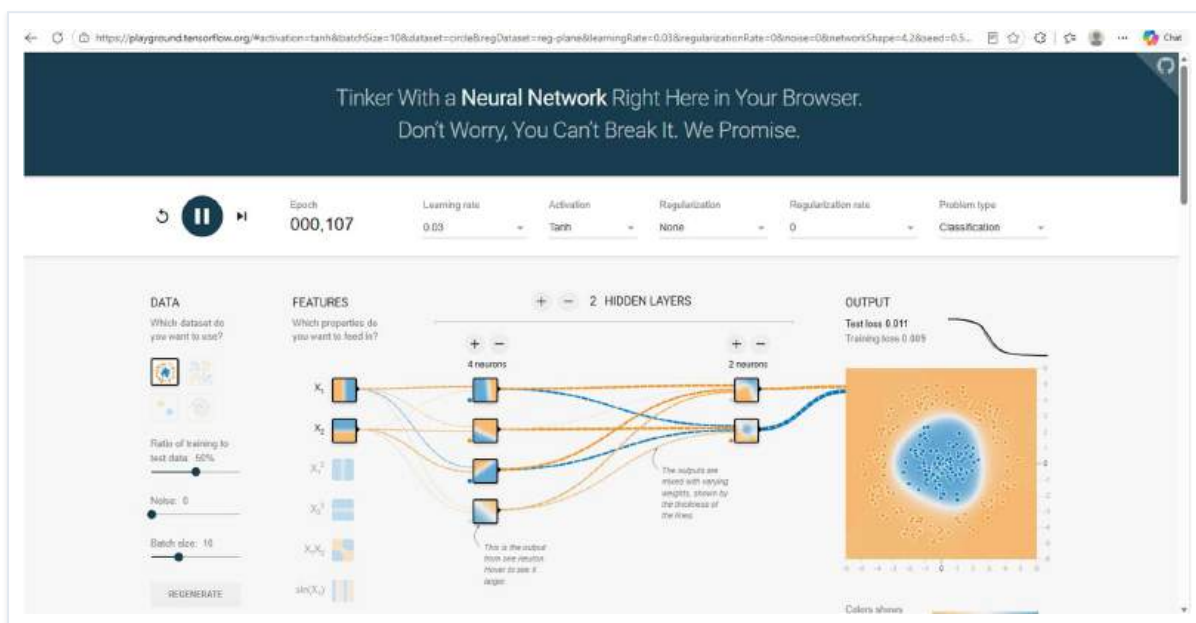


Captura 4 • Pantalla inicial de TensorFlow Playground con la red neuronal por defecto, antes de pulsar el botón de reproducción.



Iniciar el entrenamiento

Se pulsa el botón **▶ Run** y la red comienza a aprender. En cada iteración (época) ajusta los pesos de las conexiones para reducir el error. Conviene esperar aproximadamente 30 segundos y realizar una captura cuando el entrenamiento ya esté en marcha pero aún no haya convergido. En este punto se aprecia cómo el contador de épocas avanza y la gráfica de error (Test loss y Training loss) empieza a descender.



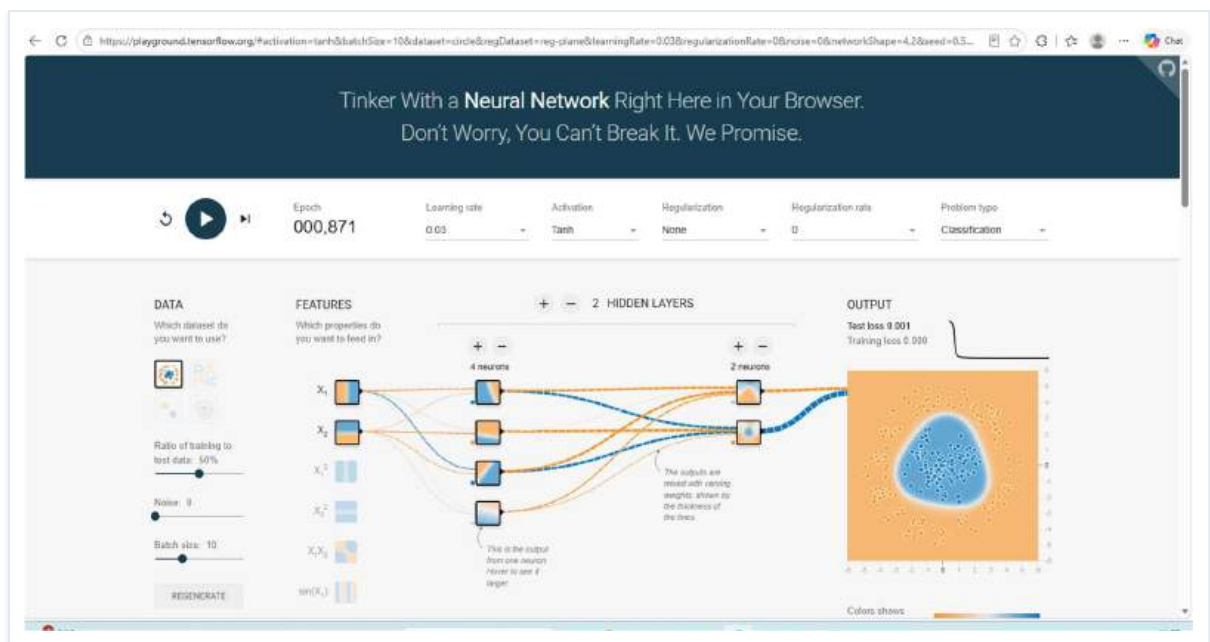
Captura 5 • Entrenamiento en marcha (época 107): los puntos de la gráfica de error ya muestran un descenso claro y la frontera de decisión comienza a adaptarse a los datos.

Observar el error, las épocas y la frontera de decisión

Se deja correr el entrenamiento y se observan tres elementos clave: el **Error (Loss)**, que indica cuánto se equivoca la red; las **Épocas (Epoch)**, que cuentan el número de iteraciones completas sobre el dataset; y la **Frontera de decisión**, el fondo de color que muestra cómo está separando la red las dos clases de puntos. Alrededor de los 30 segundos de entrenamiento, la red ya ha dado bastantes iteraciones y la frontera se ajusta casi por completo al patrón de los datos.

Pregunta: ¿Qué ocurre con el error conforme avanza el entrenamiento?

Conforme avanza el entrenamiento (es decir, a medida que aumenta el número de épocas / Epoch), el valor del error (Test loss y Training loss) va disminuyendo progresivamente hasta que se estabiliza. Al mismo tiempo, vemos cómo la frontera de decisión (el fondo de color) cambia y se adapta para envolver y separar correctamente a los puntos de un color de los del otro. La red está ajustando sus pesos matemáticos para equivocarse cada vez menos.

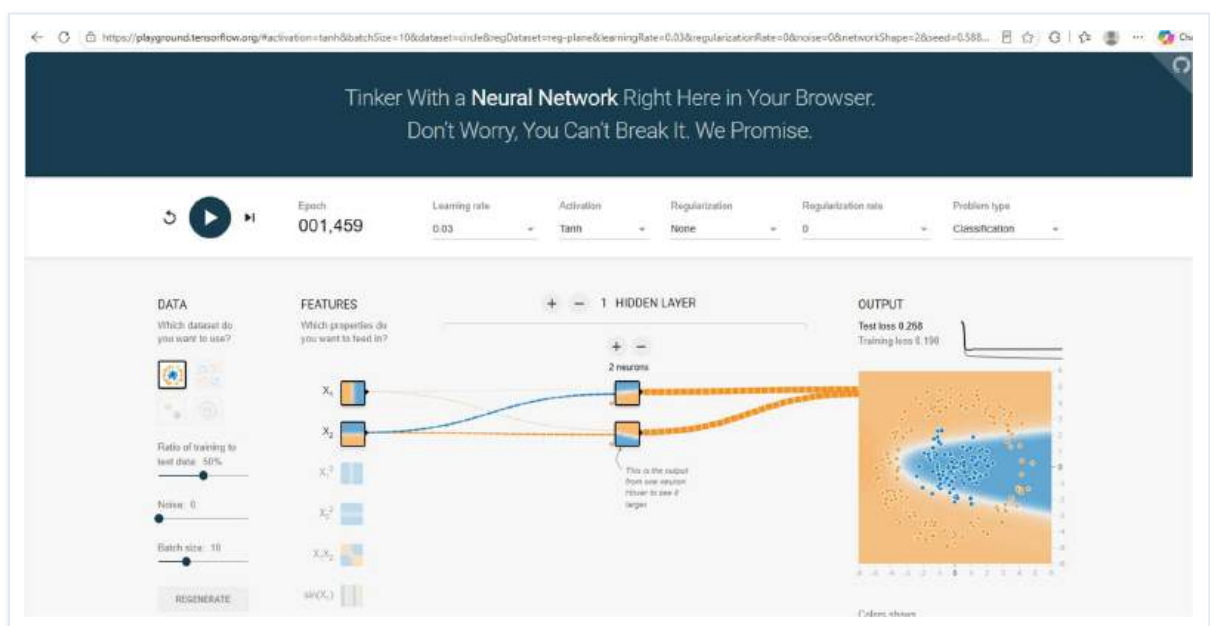


Captura 6 • Estado del entrenamiento más avanzado (época 871): el error ha descendido hasta valores muy bajos y la frontera de decisión se ajusta con precisión a los datos.

09

Modificar la red: 1 capa oculta y 2 neuronas

Se reduce drásticamente la capacidad del modelo dejando una sola capa oculta con dos neuronas. Se vuelve a pulsar **Run** y se observa el resultado: la red es demasiado simple y solo consigue trazar fronteras de decisión formadas por líneas rectas, incapaz de curvarse para envolver los datos. Es el ejemplo clásico de **subajuste**: faltan neuronas para crear las curvas necesarias.

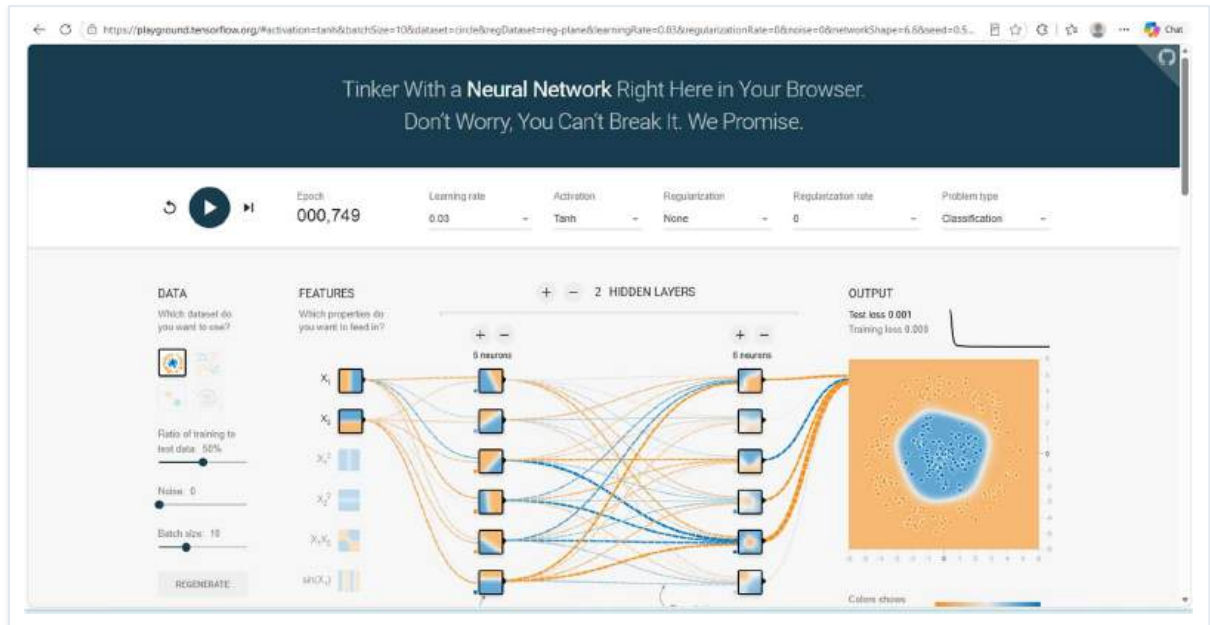


Captura 7 • Red mínima (1 capa, 2 neuronas): la frontera de decisión se reduce a una línea recta y el error se mantiene alto.



Aumentar la complejidad: 2 capas ocultas y 6 neuronas

Se amplía la red a dos capas ocultas con 6 neuronas cada una y se vuelve a ejecutar. Ahora la red sí logra adaptarse a los datos, creando fronteras de decisión complejas (círculos, espirales) que separan correctamente los puntos de cada color. El modelo tiene suficiente capacidad matemática para aprender el patrón sin llegar al sobreajuste.



Captura 8 • Red intermedia (2 capas, 6 neuronas por capa): la frontera de decisión adquiere la forma curva necesaria para separar los puntos correctamente.

Probar una red mucho más grande: 3 capas y 8 neuronas

Se configura una red notablemente mayor: tres capas ocultas con 8 neuronas por capa. Se ejecuta el entrenamiento y se observa cómo cambia la frontera de decisión. La red aprende rápido, pero al tener tanta capacidad puede empezar a "sobreaprender": memoriza el ruido del dataset en lugar de captar el patrón general y dibuja fronteras muy irregulares que no generalizarían bien a datos nuevos.



Captura 9 • Red grande (3 capas, 8 neuronas por capa): el entrenamiento es más lento por la cantidad de pesos a calcular y la frontera de decisión tiende a hacerse más irregular.

12

Comparar las tres configuraciones

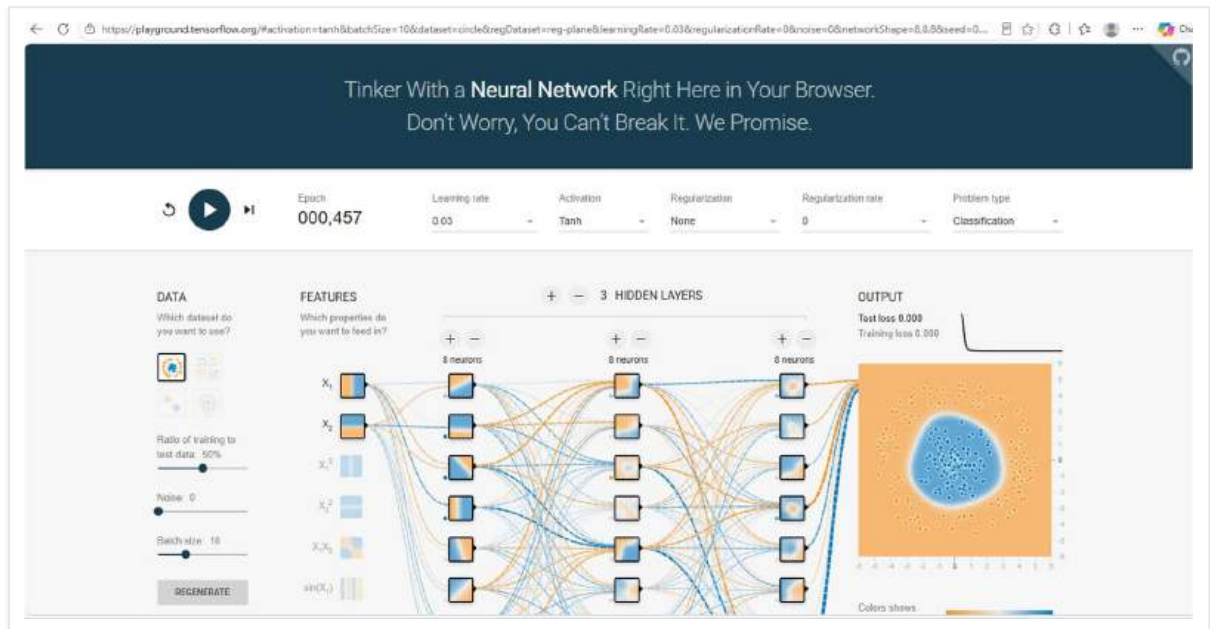
Se completa una tabla comparativa con las tres arquitecturas probadas, valorando si cada una aprende mejor y si el entrenamiento tarda más. Esta comparación permite visualizar de un vistazo el equilibrio entre **capacidad del modelo** y **coste computacional**.

Configuración	¿Aprende mejor?	¿El entrenamiento tarda más?
1 capa · 2 neuronas	No. Sufre de subajuste . Faltan neuronas para crear curvas, por lo que la red solo puede trazar líneas rectas y no logra separar bien los puntos.	No. Al tener tan pocas conexiones, los cálculos matemáticos son mínimos y las épocas avanzan a muchísima velocidad.
2 capas · 6 neuronas	Sí. Logra adaptarse perfectamente a los datos, creando fronteras de decisión complejas (como círculos o espirales) para separar los colores.	Tarda un poco más. Al haber más neuronas y capas, hay más "pesos" que calcular en cada época, ralentizando ligeramente el proceso.
3 capas · 8 neuronas	Aprende bien, pero puede "sobrepasar". Puede memorizar el ruido (padecer de sobreajuste) en lugar de entender el patrón general, creando fronteras muy irregulares.	Sí. El entrenamiento se vuelve notablemente más lento porque el número de operaciones matemáticas que debe hacer el ordenador se dispara.

13

Regenerar el conjunto de datos

Se pulsa varias veces el botón **Regenerate**: cada vez aparece un nuevo conjunto de datos con una distribución distinta de puntos (circulares, en espiral, exclusivo-OR, etc.). Se observa cómo cambia el aprendizaje con cada nuevo dataset y cómo la misma arquitectura se ve obligada a descubrir fronteras de decisión completamente distintas.



Captura 10 • Nuevo conjunto de datos generado tras pulsar **Regenerate**: la red parte de cero para aprender una frontera de decisión distinta.

14

Probar distintas funciones de activación

Se mantiene fija la arquitectura y se prueba con tres funciones de activación distintas:

ReLU

Tanh

Sigmoid

Para cada una se ejecuta el entrenamiento y se captura la pantalla. La función de activación determina cómo se transforma la salida de cada neurona y tiene un impacto enorme tanto en la forma de la frontera de decisión como en la velocidad a la que desciende el error.

RELU



Captura 11 • ReLU: fronteras formadas por líneas rectas y ángulos afilados (polígonos). Función muy rápida y eficiente computacionalmente.

TANH



Captura 12 • Tanh: fronteras muy suaves, curvas y bien definidas. Funciona muy bien para centrar los datos alrededor de cero.



Captura 13 • Sigmoid: también crea curvas suaves, pero el aprendizaje (la bajada del Loss) es extremadamente lento en comparación con las otras dos, síntoma del problema del desvanecimiento del gradiente.

Pregunta: ¿Observas diferencias entre las tres funciones?

Sí, existen grandes diferencias en cómo aprende la red visualmente:

- **ReLU:** crea fronteras de decisión formadas por líneas rectas y ángulos afilados (polígonos). Es una función muy rápida y eficiente computacionalmente.
- **Tanh:** crea fronteras de decisión muy suaves, curvas y bien definidas. Funciona muy bien para centrar los datos.
- **Sigmoid:** también crea curvas suaves, pero el aprendizaje (la bajada del Loss) es extremadamente lento en comparación con las otras dos. A veces parece que la red se queda atascada (problema del desvanecimiento del gradiente).

RESULTADO FINAL

RESULTADO FINAL

La combinación de ambas herramientas ofrece una visión completa de qué es el Deep Learning. **Quick, Draw!** demuestra el potencial de un modelo ya entrenado para inferir resultados en tiempo real, mientras que **TensorFlow Playground** permite asistir al proceso de aprendizaje y entender visualmente cómo se ajustan los pesos de la red para reducir el error. La conclusión principal es que no existe una arquitectura "perfecta": el equilibrio entre subajuste y sobreajuste depende del problema, de la cantidad de datos y de la función de activación elegida.

Aprendizaje 01

El modelo no "entiende", reconoce patrones

La IA de Quick, Draw! no tiene concepto semántico de los objetos que nombra. Convierte el trazo en vectores numéricos y calcula la probabilidad estadística de coincidencia con cada categoría conocida. La calidad del reconocimiento depende por completo del dataset de entrenamiento.

Aprendizaje 02

Más neuronas no siempre es mejor

Una red con 1 capa y 2 neuronas sufre subajuste (líneas rectas que no se adaptan). Una con 3 capas y 8 neuronas corre riesgo de sobreajuste (memoriza el ruido). La configuración intermedia de 2 capas y 6 neuronas ofreció el equilibrio óptimo entre capacidad de aprendizaje y generalización.

Aprendizaje 03

La función de activación cambia el aprendizaje

ReLU genera fronteras poligonales y entrena rápido. Tanh genera curvas suaves y bien definidas. Sigmoid también genera curvas pero sufre el problema del desvanecimiento del gradiente: el Loss baja tan despacio que la red parece atascarse en fases intermedias.

Aprendizaje 04

El error desciende hasta estabilizarse

Conforme avanzan las épocas, Training loss y Test loss descienden progresivamente. La frontera de decisión se adapta para envolver los puntos del color correcto. Cuando el error se estabiliza, la red ha aprendido todo lo que su arquitectura le permite aprender con ese dataset.

Preguntas finales

1. ¿Qué herramienta te ha parecido más sencilla de utilizar?

Google Quick, Draw!, ya que está diseñada como un juego interactivo donde el usuario solo tiene que dibujar, sin necesidad de configurar ningún parámetro matemático o técnico.

2. ¿Qué herramienta te ha ayudado más a entender el Deep Learning?

TensorFlow Playground. Permite ver exactamente qué ocurre "bajo el capó" (las capas ocultas, las conexiones, cómo cambia el error y cómo se procesan los datos geoméricamente paso a paso).

3. ¿Qué diferencias existen entre ambas herramientas?

Quick, Draw! es un modelo de "caja negra": funciona en fase de inferencia (ya sabe cómo resolver el problema y solo predice resultados) y oculta su funcionamiento interno. TensorFlow Playground es un modelo de "caja blanca": está en fase de entrenamiento (comienza desde cero) y te obliga a construir y supervisar la arquitectura de la red para que aprenda.

4. ¿En cuál de las dos herramientas había una Inteligencia Artificial ya entrenada?

En Google Quick, Draw!. Ya fue entrenada previamente por Google con más de 50 millones de dibujos.

5. ¿En cuál has podido observar cómo aprende una red neuronal?

En TensorFlow Playground.

6. ¿Por qué crees que las redes neuronales necesitan tantas capas y neuronas para resolver problemas complejos?

Porque cada capa se encarga de extraer patrones cada vez más complejos. Las primeras capas detectan cosas simples (líneas, bordes, colores básicos). Esas características simples se pasan a las siguientes capas, que las combinan para entender conceptos más complejos (formas, texturas, objetos). Sin suficientes neuronas y capas, el modelo no tiene la capacidad matemática para procesar problemas del mundo real que no son lineales.

7. Escribe tres aplicaciones reales del Deep Learning.

- **Visión Artificial y Diagnóstico Médico:** análisis automático de radiografías o resonancias para detectar tumores con mayor precisión.
- **Procesamiento de Lenguaje Natural (NLP):** asistentes virtuales y grandes modelos de lenguaje (como ChatGPT o el traductor de Google) que entienden y generan texto humano.
- **Conducción Autónoma:** sistemas en vehículos (como los de Tesla) que procesan vídeo en tiempo real para reconocer peatones, señales y otros coches, tomando decisiones al volante.

RETO OPCIONAL

RETO ADICIONAL

En TensorFlow Playground, intenta conseguir que el valor de **Loss** sea lo más bajo posible modificando:

- Número de capas ocultas.
- Número de neuronas.
- Función de activación.
- Learning Rate.

Realiza una captura de la configuración con la que hayas obtenido el mejor resultado e indica el valor mínimo de Loss alcanzado.

CONFIGURACIÓN UTILIZADA

- **Dataset:** el de la espiral (el más difícil).
- **Features:** activadas X_1 y X_2 (las coordenadas básicas) y $\sin(X_1)$ y $\sin(X_2)$ (las funciones trigonométricas de seno).
- **Número de capas ocultas:** 3
- **Número de neuronas:** 8 en la primera, 8 en la segunda y 4 en la tercera.
- **Función de activación:** ReLU (es más rápida para superar bloqueos iniciales).
- **Learning Rate:** 0,003 (si es muy alto, el modelo salta de forma caótica con esta configuración mucho más completa).

A continuación se pulsa **Play** y se deja correr el entrenamiento. En torno a la época 2000, el sistema sigue aprendiendo pero a un ritmo más ralentizado. Las dos capturas siguientes muestran el progreso: la primera en un punto intermedio del entrenamiento y la segunda con la red ya prácticamente estabilizada.

ÉPOCA 001.876



Captura 14 • Reto — Entrenamiento intermedio sobre el dataset espiral con configuración óptima.

ÉPOCA 002.484



Captura 15 • Reto — Resultado final con la frontera de decisión prácticamente ajustada a la espiral.